

★ Exercice 1: Pagination - Tables de page

On considère une mémoire paginée pour laquelle les cases en mémoire centrale sont de 2 Ko. La mémoire centrale compte au total 9 cases numérotées de 0 à 8. Dans ce contexte, on considère 2 processus A et B de tailles 8 et 16 Ko respectivement.

Q 1: Donner le format d'une adresse logique en explicitant le nombre de bits par champ sachant que l'on se place dans une machine 32 bits.

Q 2: Quelle est la taille de la mémoire virtuelle? En déduire le nombre d'entrées dans une table de page de ce système.

Q 3: Donner un contenu possible des tables de pages (case,v/i) des processus A et B ainsi que l'image de la mémoire physique correspondante. On suppose que l'on applique une allocation proportionnelle à la taille des processus A et B. On ne représente dans la table des pages que les entrées effectivement utilisées par chaque processus.

★ Exercice 2: Taille de page optimale

Soit un système dans lequel le nombre de lignes du bus d'adresse est n . On suppose que le système utilise la pagination à un seul niveau où p est la taille d'une page. Chacune des entrées de la table des pages a une taille de e octets. Déterminer la valeur optimale de p qui permet de minimiser la fonction qui cumule la perte en mémoire due à la fragmentation interne et au stockage de la table des pages.

A.N. $n = 32$ et $e = 8$

★ Exercice 3: cf. ARCHE - Pagination et traduction d'adresses

★ Exercice 4: Pagination multi-niveaux

Soit une machine 32 bits avec une taille de page de 4 Ko.

Q 4: En supposant l'utilisation d'un seul niveau de pagination, quel est le format d'une adresse virtuelle? En déduire la taille de la table des pages associée à chaque processus.

Q 5: Admettons cette fois-ci que l'on utilise une pagination à 2 niveaux et que pour chaque niveau de pagination, on attribue le même nombre de bits. On garde la même taille de cadre de page soit 4 Ko. Donner le format de l'adresse virtuelle dans ce cas ainsi que la taille globale des tables de pages. Quel est l'intérêt de la pagination multi-niveaux?

★ Exercice 5: Remplacement des pages

On considère un système à mémoire paginée où la taille d'une page est de $200 \times \text{sizeof}(\text{int})$. Un petit processus occupe la page 0 de la mémoire. Il reste 3 cadres de pages libres à l'instant où un programme arrive. Ce dernier initialise le contenu du tableau A défini par :

```
int A[] [] = new int[100][100];
```

Q 6: Sachant que ce programme est chargé dans la cadre de page numéro 1 et que les 2 autres pages restantes sont initialement libres, quel est le nombre de défauts de pages expérimenté avec un remplacement LRU et ce pour les deux programmes d'initialisation suivants :

Algorithme 1

```

1 for (int j = 0; j < 100; j++)
2   for (int i = 0; i < 100; i++)
3     A[i][j] = 0;
```

Algorithme 2

```

1 for (int i = 0; i < 100; i++)
2   for (int j = 0; j < 100; j++)
3     A[i][j] = 0;
```

★ **Exercice 6: Algorithme des frères siamois** Sous Linux, la gestion de la mémoire physique repose sur une variante de l'algorithme des frères siamois (buddy system), qui permet d'allouer et de libérer efficacement des blocs de tailles multiples de puissances de deux. Initialement, la mémoire disponible correspond à un bloc unique de 256 Ko (soit le plus grand buddy possible).

Q 7: Représenter l'état d'occupation de la mémoire après l'arrivée dans l'ordre des processus A(5K), B(25K), C(35K) et D(20K) puis leurs terminaisons dans l'ordre A, D, C et B.

Q 8: Ecrire de façon schématique la fonction trouver-trou(i) qui permet d'allouer un buddy de 2^i et retourne son adresse si possible sinon -1. Le système gère une liste pour chaque puissance de i utilisée et on ne demande pas de détailler la gestion de cette liste. 2^{max} correspond à la taille du plus grand buddy possible.